

CHAPTER 2: SOFTWARE DESIGN

This chapter presents the technical framework of the Bléum Nails By Micah Appointment System, detailing the architectural choices, engineering principles, and data processing methods employed to transition the salon from manual scheduling to an automated digital environment.

2.1 Description of the Design Solution

2.1.1 General Description

The design solution is a centralized web-based platform engineered to act as a high-performance digital bridge between customers and the salon administrator. As illustrated in the simplified diagram in Figure 2.1, the system architecture is organized into three primary stages: Input, Process, and Output. The Input Stage utilizes a client Interface accessible via mobile and desktop hardware, where users provide critical data such as service selection, appointment date and time, and digital QRPh receipts.

This data is then transmitted to the Process Stage, which serves as the central engine responsible for executing the system's core business logic, including user authentication and the Capacity-Limiting Algorithm. The processing layer maintains a continuous, bi-directional data flow with the MySQL Relational Database to securely save and retrieve client records, appointment history, and schedule availability. Finally, the Output Stage delivers real-time results back to the stakeholders, represented by a confirmed booking status for the client and comprehensive administrative analytics for the technician.

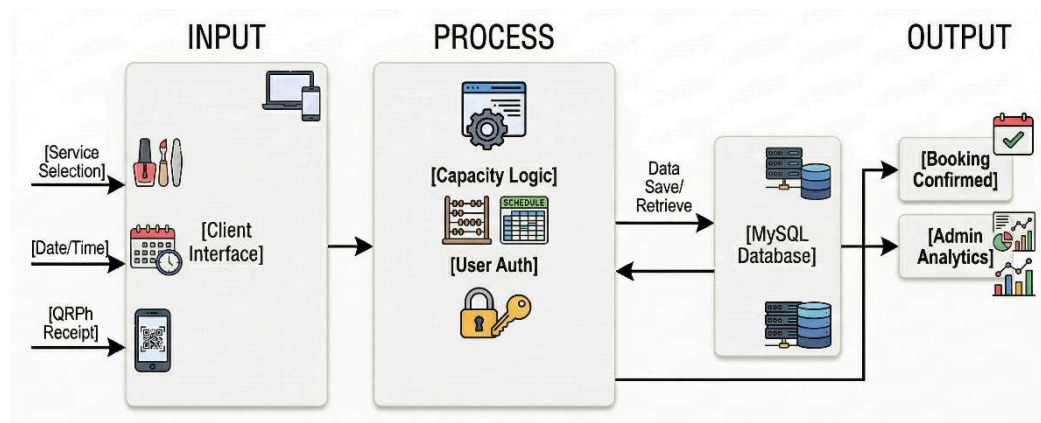


Figure 2.1 Illustrative Diagram of the System

2.1.2 Engineering Principles Involved

The development of the system is guided by foundational software engineering principles to ensure robustness and maintainability. A primary principle applied is the Separation of Concerns (SoC), where the system architecture is divided into independent layers, specifically separating the JavaScript-based User Interface from the Python-based Business Logic to allow for modular updates without causing system-wide interference. Furthermore, SOLID principles are applied to the backend code to ensure that core modules, such as payment processing and scheduling, remain highly independent and easily testable. Following empirical evidence from software development studies, the team also utilized Iterative Refinement, engaging in a continuous cycle of prototyping and testing to detect technical defects early and align the software functionality with the specific needs of the stakeholders.

2.1.3 Prior Art Analysis

To ensure unique value, the project was compared against existing commercial solutions such as Nails by Ann and Nail Garden. As shown in the matrix below, while most market tools focus on enterprise-level logistics, Bléum Nails localizes functionality through integrated QRPh payment handling and specialized Allergy Tracking modules for home-based safety.

Table 2.1 Prior Art Analysis Matrix

Design	Features					
	Online Booking System	Automated Receipts	Client Design Uploads	Allergy Tracking	Cash Payment	QRPh support
Nails by Ann	X				X	
Nail Garden	X				X	
Mich & Myl Nails					X	
PROJECT	X	X	X	X	X	X

Legend:

X = Feature Present

(Blank) = Feature Not Found or Not Available.

2.2 General System Architecture

2.2.1 Hardware Elements

Client Interface: Accessed via smartphones and personal computers through standard web browsers.

Administrator Access: Handled through the owner's dedicated computer to access the management dashboard and financial analytics.

2.2.2 Software Elements

A. Embedded Software

Not applicable, as the system is entirely web-based and does not require specialized hardware firmware.

B. Application Software

Frameworks and Languages: The backend is built on Python (FastAPI) for asynchronous processing, while the frontend is developed using JavaScript to establish a dynamic Single Page Application (SPA).

Database Management: MySQL is employed using relational modeling and normalization to ensure the integrity of client and appointment records.

C. Key Algorithms Used (Optional)

A central component of the system is the **Capacity-Limiting Algorithm**, which automates the enforcement of a three-client daily maximum to prevent operational fatigue.

- **Input:** The system receives a booking request for a specific target date.
- **Query:** A SQL count query is executed against the database to identify confirmed bookings for that date.
- **Validation:** The algorithm compares the current booking count to a constant value of 3.
- **Action:** If the count is less than 3, the slot is marked available; if it is 3 or more, the system triggers a "Fully Booked" status and restricts further selection for that date.

2.2.2 System Algorithm

The main system algorithm manages the flow of a booking from start to finish:

1. **Availability Check:** The system checks the database for available dates.

2. **Booking Validation:** It verifies that the daily limit of three clients has not been reached before confirming a new slot.
3. **Payment Processing:** Upon service completion, the algorithm tracks whether the payment was confirmed or still on process.

2.2.3 Data, Datasets, and Processing

a. Datasets

The system utilizes three primary categories of data to ensure operational functionality and testing accuracy:

- **User-Generated Data:** This includes real-time information acquired from clients through registration and booking forms, such as names, emails, and phone numbers. The acquisition process is handled via Pydantic models that validate raw HTTP POST requests before they are committed to the database.
- **Administrative Managed Data:** The system maintains a curated dataset of salon services and gallery items. This includes service names, price constants (e.g., ₱199 for Gel overlays), and metadata for the artist's portfolio.
- **Third-Party OAuth Data:** Profile information (email and names) is acquired from the Google OAuth 2.0 API when users opt for social login. This data is used to auto-populate the Accounts table, ensuring a frictionless user experience.

b. Data Processing Scheme and Algorithms

The system follows a strict processing pipeline to transform raw user input into secure, structured records:

1. **Raw Input Layer:** Data is received as unstructured JSON objects from the frontend (e.g., a plain-text password or a raw payment ID).
2. **Pre-processing and Validation:** The system utilizes Pydantic to enforce data types and CryptContext (bcrypt) to hash raw passwords. This ensures that sensitive credentials never exist as plain text within the system.
3. **Core Business Logic Application:** The Capacity-Limiting Algorithm processes the target date of a booking request. It executes a select statement against the SQLite database to verify the current booking count for that specific date.
4. **External Logic Integration:** For financial data, the system processes PayMongo webhook payloads. It converts Unix timestamps into ISO-formatted dates and verifies payment IDs before updating an appointment status from "Awaiting Payment" to "Confirmed".

5. **Final Dataset:** The output is a normalized relational record stored in database.db, adhering to SQLModel standards for long-term data integrity.

c. Other Data Utilized in the Design

Beyond active user records, the system relies on specialized data structures to maintain security and configuration:

- **Service Constants:** Fixed values stored in the Services table used for automated receipt generation and balance calculation.
- **Environment Variables (.env):** Secured data such as SECRET_KEY, GOOGLE_CLIENT_ID, and ADMIN_ACCOUNTS string. This data is used by the JWT engine to sign session tokens, ensuring that only authorized users can access protected dashboards.
- **Email Templates:** Pre-formatted HTML/String data used by the SMTP server to generate verification codes and booking confirmations.

2.3 Design Alternatives

2.3.1 Rationale for Design Alternatives

The development of the Bléum Nails By Micah Appointment System is proposed to address the operational inefficiencies associated with manual scheduling, such as handwritten logs and fragmented messaging. These manual processes often result in asynchronous scheduling lag, leading to data integrity errors and double-booked time slots. By providing real-time access to technician availability and implementing a structured digital framework, the proposed system seeks to streamline the booking process, improve technician efficiency, and enhance client trust.

To ensure a professional and secure environment, the system is designed in compliance with ISO/IEC 25010 software quality standards and RA 10173 (Data Privacy Act). The following architectural alternatives were selected to identify the most balanced solution for a home-based business, prioritizing a mobile-first responsive approach to accommodate the high volume of users accessing services via smartphones.

2.3.1 Design Alternative 1: Monolithic Server-Side Architecture

Tech Stack: HTML, CSS, Python (Django/Flask)

A. Engineering Principles of Alternative

Server-Side Rendering (SSR): The server generates the full HTML response for every request, reducing client-side processing requirements.

Relational Data Modeling: Employs a structured SQL database with normalization (3NF) to ensure strict data integrity for appointment records.

Encapsulation: All business logic, routing, and data access are contained within a single application boundary for simplicity.

B. Architecture of Design Alternative

This follows Monolithic Architecture. The system operates as a single unit where the Python backend directly queries the database and renders templates (HTML/CSS) to the user's browser. It is a "Request-Response" cycle where the server is the primary engine of the application.

C. Constraints

Usability: High for desktop users; however, frequent page reloads may feel "clunky" on modern mobile devices.

Functionality: Capable of handling core booking tasks, though limited by the lack of dynamic background updates.

Performance: Moderate; every interaction requires a round-trip to the server, resulting in higher latency per click.

Reliability: Very High; the logic is centralized and executes consistently if the server is operational.

Maintainability: Very High; a single codebase in Python is easier for a solo developer to debug and update.

2.3.2 Design Alternative 2: Decoupled Full-Stack Architecture

Tech Stack: HTML, CSS, JavaScript, Python (FastAPI/REST)

A. Engineering Principles of Alternative

Separation of Concerns: Distinct separation between the User Interface (JavaScript) and Business Logic (Python).

Asynchronous Communication (AJAX/Fetch): The UI updates dynamically by fetching data in the background without reloading the page.

Statelessness: The backend acts as a RESTful API, improving scalability and making the system more modular.

B. Architecture of Design Alternative

This is a Client-Server Architecture. The front end is a Single Page Application (SPA) built with JavaScript that runs in the browser. It communicates with a Python-based API via JSON. This allows for a very responsive, "app-like" experience.

C. Constraints

Usability: Extremely High; provides fluid transitions and a modern feel that users prefer.

Functionality: High; supports complex real-time logic such as the Capacity-Limiting Algorithm more efficiently than a monolith.

Performance: High; initial load might be slower, but subsequent interactions are near instant.

Reliability: High; however, client-side state can be lost if the browser or network "hiccups".

Maintainability: Medium; requires managing two separate environments (Front and Back).

2.3.3 Design Alternative 3: Enterprise-Tier Architecture

Tech Stack: Java (Spring Boot) gr, PostgreSQL, HTML, CSS

A. Engineering Principles of Alternative

Strong Typing and Type Safety: Java's strict typing reduces runtime errors and is ideal for complex, multi-user scheduling logic.

Persistence Excellence: PostgreSQL provides advanced concurrency control for multiple users booking at the exact same time.

Multi-threading: Spring Boot can handle many simultaneous requests more efficiently than standard Python configurations.

B. Architecture of Design Alternative

This follows Enterprise Layered Architecture. It includes a Controller layer, Service layer, and Repository layer. This is built for high-scale environments where security and data concurrency are the top priorities.

C. Constraints

Usability: Moderate; focuses more on robust data handling than flashy UI transitions.

Functionality: Very High; capable of handling enterprise-level logistics and multi-user concurrency.

Performance: High; excellent at handling heavy data loads and large databases.

Reliability: Highest; Java's ecosystem is built for "mission-critical" stability and uptime.

Maintainability: Low; the codebase is often "verbose," requiring significantly more lines of code and longer development time.

2.4 Standards Involved in the Design

The following matrix presents the mapping of specific engineering standards to the three design alternatives, justifying the technical robustness of each architecture based on industry-recognized benchmarks

Table 2.2 Summary of Standards Involved in the Alternatives

Standard	Brief Description	DESIGNS		
		Monolithic Server-Side	Decoupled Full-Stack	Enterprise-Tier
ISO/IEC 25010	International quality model for software products focusing on functional suitability and usability.	Used for performance and maintainability checks		
Republic Act No. 10173	Primary law in the Philippines protecting individual personal information in digital systems.	Applied to ensure legal protection of client contact and other personal information.		
W3C Standards	International benchmarks for web development ensuring cross-browser compatibility.	Applied for standard desktop and mobile layouts.		
PCI DSS	Standards for secure third-party payment integration to protect financial transaction records.	NA	Applied for secure localized payment (QRPh) and receipt verification.	Applied for enterprise-tier secure payment integration.
OWASP ASVS	Standard for testing and verifying web application security controls.	Focused on secure login and authentication.		

iCalendar (RFC 5545)	Standard for representing calendar events and scheduling data.	Used for simple appointment scheduling
-------------------------	---	--

The table above presents the mapping of specific engineering standards to the three design alternatives. This matrix is used to justify the technical robustness of each design based on industry recognized benchmarks:

- **Quality and Maintainability:** ISO/IEC 25010 is applied specifically to all designs to ensure that the system meets strict performance and maintainability checks. This ensures the software remains reliable under consistent use by the salon staff.
- **Data Privacy Compliance:** Republic Act No. 10173 (Data Privacy Act of 2012) is applied across all architectural options to ensure the legal protection of client contact details and personal information. Adhering to this standard ensures that sensitive records are handled with the highest level of confidentiality.
- **Web Architecture:** All front-end development for all designs adheres to W3C HTML5 / CSS3 standards to guarantee cross-browser compatibility and a responsive mobile-first layout.
- **Financial Security:** The PCI DSS standard is applied specifically to Design B and Design C to support secure localized payment (QRPh) and receipt verification. This protects the financial integrity of the salon's transaction records, a feature that is not available (NA) in the simpler monolithic Design A.
- **System Security and Interoperability:** Design C prioritizes high-level security by implementing the OWASP Application Security Verification Standard (ASVS) for secure login and authentication. Additionally, Design C incorporates the iCalendar (RFC 5545) standard for simple appointment scheduling and interoperability with external applications.

In summary, the integration of these engineering standards serves as a foundational framework for the design and development of the online booking platform. By adhering to ISO/IEC 25010, RA 10173, W3C, and PCI DSS protocols, Team Sentinails ensures that the final product particularly the selected Design B is not only a functional scheduling tool but a secure, reliable, and high-quality software solution. These standards collectively bridge the gap between manual booking and automated efficiency, fulfilling both the client's needs and professional engineering requirements.